A Critique of Modern SQL And A Proposal Towards A Simple and Expressive Query Language

Thomas Neumann
Technische Universität München
neumann@in.tum.de

Viktor Leis
Technische Universität München
leis@in.tum.de

ABSTRACT

The first contribution of the paper is a comprehensive critique of modern SQL, informed by an analysis of real-world SQL queries. This provides the motivation for our second contribution: the *Simple ANd Expressive Query Language (SaneQL)*. SaneQL features a straightforward and consistent syntax, which improves its learnability and ease of implementation. Additionally, it provides extensibility, with the added ability to define new operators that integrate seamlessly with the existing built-in ones. Unlike most data frame APIs and NoSQL query languages, SaneQL fully embraces the core principles behind SQL, especially multiset semantics. We propose that adopting SaneQL's approach can ensure the enduring success of relational database technology, offering the power of SQL's underlying concepts through a more accessible and flexible language.

1 INTRODUCTION

SQL. Despite celebrating its 50th anniversary in 2024 [4], SQL is still *the* predominant query language. Its success is inextricably linked with the success of the relational model. In comparison with other languages, it stands out for its declarative nature, multiset semantics, and three-valued logic. These concepts have stood the test of time and enable data independence, effective query optimization, and automatic parallelization.

Problem 1: Irregular Pseudo-English Syntax. Unusually among widely-used programming languages today, SQL has an English-inspired syntax. The motivation behind the surface syntax of SQL stems from the desire to make queries easy to read. SQL's inventor Don Chamberlin calls this the "walk up and read" property [2]. It is true that the meaning of a query like `SELECT name FROM customer WHERE id = 42` can indeed be guessed without any formal training. However, this is only true for simple queries, and optimizing for this kind of readability comes at great cost. The irregularity of modern SQL, which has grown tremendously over the past five decades, makes it hard to learn, cumbersome to write, difficult to debug, hard to implement, and leads to impenetrable error messages. The dominance of SQL may be at risk, as evidenced by the rising popularity of data frame APIs, such as Python's Pandas. Such APIs have, to a significant extent, already replaced SQL in data science.

Problem 2: Lack of Extensibility and Abstraction. The cornerstone of any powerful programming language is a mechanism for abstraction – and SQL is lacking severely on this front. SQL offers views (and their transient variant Common Table Expressions) but these merely allow naming and re-using parts of a query. For example, it is not possible to pass a relation into a view definition as argument. This effectively makes SQL a functional programming language that has functions *without* parameters. More advanced features such as passing expressions are obviously not supported either. For example, imagine implementing a semi join or pivot operator that works for arbitrary input relations and join predicates. This is simply not possible in SQL. SQL fundamentally violates the "don't repeat yourself" principle in software engineering, and instead relies on sheer repetition.

Contributions. This paper makes two contributions. First, Section 2 provides a detailed critique of modern SQL. We do this through example queries that highlight SQL's weaknesses, and through the analysis of real-world SQL queries that show the difficulties SQL learners face. Our critique provides the rationale for the second contribution, the *Simple ANd Expressive Query Language (SaneQL)*, which we introduce in Section 3. SaneQL has a simple and regular syntax, which not only makes it easier to learn and implement, but also enables extensibility. In SaneQL, it is possible to define new operators that are indistinguishable from built-in ones. In contrast to most data frame APIs and NoSQL query languages, SaneQL fully embraces the core ideas behind SQL, in particular multiset semantics. We believe that SQL has become successful due to its powerful underlying concepts rather than its unusual syntax – and that adopting a new modular surface syntax is the best way to ensure the enduring success of relational database technology.

2 A CRITIQUE OF MODERN SQL

Query Dataset. In this section, we critique modern SQL through query examples that illustrate its irregular nature. Additionally, we provide empirical evidence for the claim that SQL is hard to learn through a real-world query dataset. We collected 130,998 queries from the https://hyper-db.com website in January 2019. The website provides a web-based SQL interface and is primarily used by students learning SQL at the Technical University of Munich (TUM) and other universities. The final exam for the TUM *Introduction to Databases* undergraduate course, which has over 1,000 students, takes place in February. Thus, the dataset reflects the difficulties that SQL learners preparing for an exam have, rather than the difficulties experienced users face. The first striking result is that out of all queries, 50,683 (38%) result in an error when executed. The vast majority of these errors are compile time (not runtime) errors. While some of these are clearly unavoidable (e.g., incomplete queries), we believe that many cases would be unnecessary with a simpler, more regular query language.

This paper is published under the Creative Commons Attribution 4.0 International (CC-BY 4.0) license. Authors reserve their rights to disseminate the work on their personal and corporate Web sites with the appropriate attribution, provided that you attribute the original work to the authors and CIDR 2024. 14th Annual Conference on Innovative Data Systems Research (CIDR '24). TODO, Chaminade, USA

Table 1: Error messages of 50,683 invalid SQL queries from https://hyper-db.com website.

%	error message (in PostgreSQL)
39.8	syntax error at ...
16.6	column ... does not exist
9.0	column ... must appear in the GROUP BY clause
7.2	relation ... does not exist
5.7	missing FROM-clause entry for table
4.0	subquery in FROM must have an alias
2.2	division by zero (runtime error)
2.2	column reference ... is ambiguous
1.9	aggregate functions are not allowed in ...
1.5	operator does not exist
1.4	each UNION query must have same number of columns
1.2	function does not exist
0.9	invalid reference to FROM-clause entry for table
0.6	SELECT * with no tables specified is not valid
0.5	aggregate function calls cannot be nested

2.1 Syntax

Unhelpful Error Messages. Table 1 shows the most common error messages as reported by PostgreSQL. Many of the most common error messages are not very helpful, and usually do not explicitly say what the conceptual problem is or how to fix it. This is particularly true for *syntax errors*, which, as Table 1 shows, are by far the most common error type. As we discuss below, the underlying root cause is often irregular grammar leading to spurious commas and misplaced parentheses.

WITH. A construct that is surprisingly difficult to get right is Common Table Expressions (CTEs). Semantically, Common Table Expressions without recursion are straightforward: they define one or more transient views that can then be referenced like any base table. When teaching SQL, we recommend using the `WITH` construct to decompose larger queries into intermediate steps. The correct syntax is as follows:

```
WITH r AS (SELECT 1), s AS (SELECT 2) ...
```

The following variants are illegal (and it's easy to construct more):

```
WITH r AS (SELECT 1) s AS (SELECT 2) ...
WITH r AS SELECT 1, s AS SELECT 2 ...
WITH (r AS SELECT 1), (s AS SELECT 2) ...
WITH r AS (SELECT 1), s AS (SELECT 2), ...
WITH r AS (SELECT 1) WITH s AS (SELECT 2) ...
```

We have encountered many similar errors in our query dataset. Overall, only 45% of all queries containing `WITH` are correct. Finally, as an example of SQL's limited syntactical orthogonality, observe that in the context of WITH one has to write `WITH name AS value`, while in the `SELECT` clause one writes `SELECT value AS name`.

WITH RECURSIVE. SQL also offers recursive common table expressions, which appear as a minor syntactic variation of non-recursive CTEs, but, by making the language Turing complete, are much more powerful. The following example computes the Fibonacci sequence:

```
WITH RECURSIVE fib(a,b) AS
 (SELECT 0, 1 -- base case
  UNION ALL
  SELECT b, a+b -- recursive case
  FROM fib
  WHERE a<100)
SELECT a FROM fib
```

In most programming languages, recursion is a powerful and general construct. SQL's recursive CTEs, on the other hand, are highly restricted: the SQL standard only permits referencing the recursive relation ("fib" in the example) once, which prevents many interesting computational use cases [1]. This limitation implies that the construct is semantically equivalent to iteration rather than recursion. Indeed, the best way to understand the semantics of a WITH RECURSIVE query is through the actual iterative execute logic of the construct: it first executes the base case, then iteratively executes the "recursive" case using the prior result as input until it returns an empty result. We argue that specifying this process as a recursion often makes formulating complex computational logic unnecessarily abstruse and hard to understand.

VALUES. Another syntactically difficult SQL construct is the construction of constant inline tables, which uses the following syntax:

```
SELECT * FROM (VALUES ('x',5), ('y',2)) AS r(a,b)
```

Note the specific locations of the four pairs of parentheses. The following three variants result in a syntax error:

```
SELECT * FROM VALUES ('x',5), ('y',2) AS r(a,b)
SELECT * FROM (VALUES ('x',5), ('y',2) AS r(a,b))
SELECT * FROM (VALUES (('x',5), ('y',2))) AS r(a,b)
```

Given this unforgiving syntax, we are not surprised that students find it difficult to write correct VALUES expressions. In our dataset only 40% of all queries that contain the `VALUES` construct are correct.

Verbose Base Table Queries. The simplest (and maybe most common) query is probably `SELECT * FROM table`. This raises the question why it is not sufficient to simply write `table`. A related question is why one has to write `SELECT * FROM r UNION SELECT * FROM s` rather than `r UNION s`.

Too Many Keywords. The SQL parser relies heavily on reserved keywords. SQL-92 defined 227 keywords, and over time this number increased, arriving at 409 with SQL:2023[1]. These 401 keywords include many common English terms, and represent 18% of all English word usage[2]. It is therefore not surprising that, in our query dataset, a significant fraction of invalid table and attribute accesses stem from reserved keywords being used as identifiers. The SQLite documentation comments on this situation as follows: "The list of keywords is so long that few people can remember them all. For most SQL code, your safest bet is to never use any English language word as the name of a user-defined object."[3]

[1] https://www.postgresql.org/docs/current/sql-keywords-appendix.html
[2] This was computed using English word frequency data from https://wortschatz.uni-leipzig.de/en/download/English.
[3] https://www.sqlite.org/lang_keywords.html